

《人工智能通识》

第十讲: AI 能力选择指南——从需求到方案的完整决策路

周天航

重质油全国重点实验室
中国石油大学 (北京) 未来能源学院

2026 年 9 月



先回顾: 第九讲的微调, 三句话还记得吗?

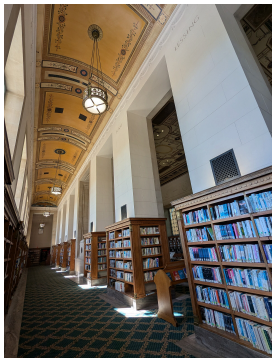
- Q1** 微调改的是什么? 改**参数**, 不是改输入; 通才变专才
- Q2** 什么时候才该微调? Prompt 和 RAG 都试过, **风格 / 格式**仍不稳定
- Q3** 微调不能解决什么? **天天变**的知识、**实时**的数据、跨轮**记忆**

九讲下来, 你手里已经有了一整套“工具”:

Prompt · RAG · Function Call · MCP · Agent · 上下文工程 · 微调

但只会用每一件工具, 还不等于会做一个项目——
今天这一讲, 把它们从“知识”整合成**一套决策方法**。

情境：实验室的一次组会



老师：“我想要一个 AI, 能回答我们组文献里的问题。”

同学 A “写个好 Prompt 就行了吧?”

同学 B “得上 RAG, 我第四讲学过。”

同学 C “要不直接微调一个化工模型?”

同学 D “做个 Agent, 让它自己去查!”

Q: 四个人说的都是学过的东西, 到底谁对?

Q: 如果老师又加一句“还要能看今天的反应器数据”, 答案会变吗?

2000 篇重油加工 PDF, 老师想要
一个“文献助手”

一句常被引用的话: 企业落地 AI 最大的失败原因不是技术不行,
而是**需求没有被正确拆解, 能力没有被正确选择。**

选错能力, 项目会变成什么样?

常见错法	后果
采用了与需求不匹配的能力	方案 过度复杂 , 几小时能搭好的东西拖成几个月
忽略了必须要用的能力	效果 不稳定 , 演示能过, 上线就翻车
在 RAG 和微调之间纠结	知识天天变却去微调, 一上线就过时
在 Agent 和 Workflow 之间迷茫	本该按流程走的事让模型“自由发挥”

今天你将带走的东西:

1. 一套 **需求** → **能力** → **方案** 的决策体系 (六问 + 六步决策树);
2. 一套从 0 到 1 的 AI 系统建设路径 (由轻到重, 先跑通再加重);
3. 五个化工需求的**方案配方**, 可以直接拿去做期末项目。

本讲学习目标

知识目标

说清 AI 能力的三层结构 (即时 / 外置 / 内置) 及各层代价; 记住需求拆解的六个问题和六步决策树; 知道大小模型、开源闭源、推理模型各适合什么场景。

能力目标

拿到一个具体需求 (尤其是化工场景), 能填出需求卡、走完决策树, 给出 “Prompt + RAG + ...” 的能力组合和分阶段建设路径, 并说明每一步为什么需要。

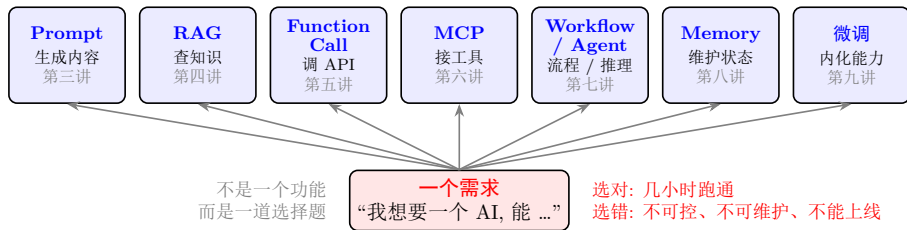
素养目标

建立 “够用就好、由轻到重” 的工程判断; 对 AI 输出保持核对习惯; 处理厂内数据时先想合规与安全。

学习建议: 每讲到一个能力, 先在心里回答 “它是哪一讲讲过的, 解决什么问题, 代价是什么”。

第一部分·大模型不是一个功能, 而是一整套能力生态

不同层级·不同代价·不同边界——同一个需求可能有 6 ~ 7 种解法



同一个需求, 可能有 6 ~ 7 种完全不同的解法——听起来强大, 其实危险:

大模型不是“功能”, 而是“选择题”。选错能力, 项目就会不可控、不可维护、不能上线。

五个典型误区, 换成化工场景你可能都见过

常见的误区	化工版
本来只要 Prompt, 却硬搞 RAG	让 AI 把一段实验记录改成规范的报告语言, 却先花两周建知识库
明明需要查数据库, 却让模型“猜”	问“三号反应器现在多少度”, 模型凭训练记忆编一个数
知识每天变, 却去做微调	把本周的原油性质数据微调进模型, 下周原油换了
本该用 Workflow, 却上 Agent	实验报告固定五段, 却让 Agent 自己决定写什么
本该微调, 却反复堆 Prompt	要求 500 份报告格式完全一致, 靠 Prompt 总有 5% 跑偏

这些问题的本质不是“大模型不行”, 而是**没有看见它的能力结构**。

举手投票：凭直觉先选一次

课堂互动①

下面三个需求，每个只能选一种能力：A. Prompt B. RAG C. Function Call D. 微调。

每题举手 10 秒，记下全班的分布；本讲结束前我们再投一次，看有没有变化。

1. 把一段口语化的实验记录，改写成实验报告的规范表述；
2. 问“我们实验室安全手册里，使用氢氟酸的防护要求是什么”；
3. 问“常减压装置常压塔塔顶温度现在是多少”。

先别急着对答案。三道题分别落在三个不同的能力层上——这正是下一页那张金字塔要说的事。

第一层：即时能力——通过指令引导，解决约 60% 的问题

Prompt	合理构造任务描述，让模型按预期方式输出	第三讲
Few-shot	给几个示例，让模型快速理解任务模式	第三讲
Chain-of-Thought	引导模型一步步推理，提高复杂问题准确率	第三讲

优势

零成本、即时生效、快速迭代、无需技术门槛

劣势

效果上限有限、结果不稳定，每次调用都要传完整指令

适用：通用对话、文本生成、快速验证、MVP 开发。

化工例子：改写实验记录、把英文文献摘要翻成中文、按模板生成周报
——**这一层就够了。**

第二层: 外置能力——连接外部资源, 解决剩余约 30%

RAG	结合知识库实时检索, 补充最新与专业知识	第四讲
Function Call	调外部 API 或工具执行操作, 增强执行力	第五讲
MCP Tools	标准化接口统一管理外部能力, 便于维护扩展	第六讲
Workflow	编排多步骤操作, 实现复杂业务流程自动化	第七讲

优势

灵活可更新、成本可控、来源可追溯

劣势

依赖外部系统、响应速度受影响、要维护接口

适用: 知识问答、数据查询、自动化任务、跨系统集成。

化工例子: 安全手册问答 (RAG)、读 DCS 位号实时值 (Function Call)、按 SOP 生成开车方案 (Workflow)。

第三层：内置能力——改变模型内核，成本最高

微调 Fine-tuning	用领域数据再训练参数，内化专业知识与风格	第九讲
蒸馏 Distillation	把大模型能力迁移到小模型，降低推理成本	第九讲
对齐 Alignment	用 RLHF 等技术让输出更符合人类偏好	第一讲

优势

效果最稳定、响应快、不依赖外部系统

劣势

成本极高、更新困难、需要大量标注数据

适用：垂直领域应用、统一语气与格式、需要固化的能力。

第九讲的原话：**微调不是万能药，而是“不可替代”的认知重塑**——只在核心业务或长期战略中用。

第二部分·先别选能力, 先把需求问清楚: 六个问题

①任务类型

生成 / 提取 / 问答
操作 / 推理

②数据在哪

模型脑子里 / 文档
业务系统 / 哪儿都没有

③实时性

几年不变 / 按月更新
按秒变化

④准确率

可容错 / 要出处
零容错

⑤成本预算

几十元 / 几千元
几万元起

⑥安全合规

数据能否出厂
会否触发动作 / 谁负责

需求原话

“我想要一个 AI ...”

一张需求卡
六个格子都填上

再谈技术

能力组合 (第三部分)

能力选错, 十有八九是因为**需求没拆开**。

拿到任何一个“我想要一个 AI ...”的需求, 先把这六个格子填满, 再谈用什么技术。

问题一：任务属于哪一类？(它决定主线能力)

类型	化工里的例子	常用能力
生成类	写实验报告初稿、翻译文献摘要、写周报	Prompt
提取类	从 50 篇论文里抽出催化剂、温度、转化率	Prompt + Few-shot
问答类	“氢氟酸泄漏应急处置流程是什么”	RAG
操作类	查 DCS 位号、下载物性数据、发送报警单	Function Call / MCP
推理类	“温升偏快是催化剂失活还是进料变了”	Workflow / Agent

一句话记住：如果你不先判断任务类型，你一定会选错能力。

一个真实需求往往同时含两三类——那就拆成两三个子任务，分别选能力。

问题二、三：数据在哪？要多“新”？

数据在哪

- ▶ **模型脑子里**：通用知识、教科书级化工原理 → Prompt 就够
- ▶ **一堆文档**：安全手册、SOP、组内文献 → RAG
- ▶ **一个系统**：DCS、LIMS、ERP、物性数据库 → Function Call / MCP
- ▶ **哪儿都没有**：得先造数据，再谈微调

实时性

- ▶ **几年不变**：化工原理、单位换算 → 模型自带
- ▶ **按月 / 周更新**：规程版本、文献库 → RAG, 换文档即可
- ▶ **按秒更新**：塔顶温度、泵出口压力 → 必须现查, 任何“记住”的方式都不行

一条经验：**数据更新得越快，越不能塞进模型参数里**——微调进去的东西，改起来最慢。

问题四、五：错了会怎样？预算有多少？

准确率要求

- 可容错** 文献综述初稿、头脑风暴：错了人再改
- 要出处** 安全规程问答：每句话能点开原文核对
- 零容错** 联锁参数、投料量：AI只能**建议**，不能**执行**

成本预算

- 几十元** 调 API + Prompt，一下午跑通
- 几千元** 建向量库、接接口、写 Workflow，一两周
- 几万元起** 标注数据 + 微调 + GPU，以月计 (第二讲的算账方法还记得吗)

准确率要求越高，越要能**追溯来源**、越要有**人在回路**；

预算越少，越要老老实实从金字塔**底层**开始。

价格为数量级示意，不同模型、不同规模差别很大。

问题六：安全与合规——化工场景绕不开的一问



工艺包、配方、DCS 数据都是厂里的核心资产

先问三件事：

1. 这些数据**能不能出厂**？配方、工艺参数、事故记录通常不能上公有云 API；
2. 输出会不会**直接触发动作**？能改设定值的接口，必须加人工确认；
3. 出了错**谁负责、怎么追溯**？要留调用日志和来源引用。

这一问会直接改变模型选型：

数据不能出厂 ⇒ 用**开源模型本地部署** (如 Qwen、DeepSeek 开源版, 可从 hf-mirror.com 下载), 而不是调云端 API。

合规不是最后再补的一步，它在第一天就决定了你能用什么。

小组：给“DCS 异常解释助手”填需求卡

课堂互动②

需求原话：“值班工程师问一句‘三号加氢反应器今晚温升为什么偏快’，AI 给出解释和建议。”

4 人一组，3 分钟，在草稿纸上画六个格子，把六个问题的答案填进去；然后两组交换，挑对方漏掉的约束。

填的时候想清楚：

- ▶ 这句话里其实藏了几类任务？（查数 + 找历史工况 + 推理解释 ...）
- ▶ “今晚”两个字，对实时性意味着什么？
- ▶ 如果 AI 说“建议把进料量降 5%”，谁来按下按钮？

挑错标准：有没有漏掉“数据能否出厂”和“零容错”这两条。

需求卡参考答案: DCS 异常解释助手

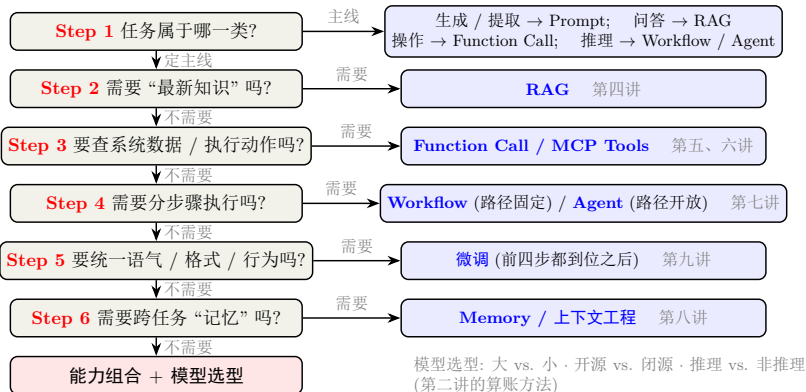
任务类型	操作类 (读位号) + 问答类 (查历史工况、规程) + 推理类 (解释原因)
数据在哪	DCS 实时库 (系统)、历史工况与事故报告 (文档)、工艺原理 (模型自带)
实时性	温度、流量按秒更新, 必须现查; 历史工况按月更新
准确率	解释可容错 (工程师会判断), 但引用的数据和规程必须可核对
成本预算	中: 接 DCS 接口 + 建工况知识库, 不需要微调
安全合规	数据不出厂, 本地部署; AI 只给建议, 不能写设定值; 全程留日志

填完这张卡, 你会发现能力组合几乎已经“自己长出来了”:

Function Call (读 DCS) + RAG (历史工况) + Workflow (先查数、再找相似工况、再解释) + 本地开源模型

Q:有同学选了微调吗? 六问里哪一条否决了它?

第三部分 · 六步决策树: 能解决 90% 的需求



六步走完, 基本就能回答: **我应该用什么能力来做这个项目?** 每一步都对应前面某一讲。

Step 1: 任务属于哪一类? ⇒ 决定主线能力

任务类型	主线能力	判断口诀
生成类	Prompt	输出是“写出来”的, 素材模型自己有
提取类	Prompt + Few-shot	输入长、输出短且结构化, 给两个例子就会
问答类	RAG	答案在 某份文档 里, 而不是在模型脑子里
操作类	Function Call / MCP	答案在 某个系统 里, 或者要“做一件事”
推理类	Workflow / Agent	要 多步 、要判断、要根据中间结果决定下一步

“哪一讲讲过”: 生成 / 提取 → 第三讲; 问答 → 第四讲; 操作 → 第五、六讲; 推理 → 第七讲。

主线定了, 后面五步只是往主线上“加配件”。

Step 2: 需要“最新知识”吗? $\Rightarrow$ RAG

出现下面任何一条, 就需要 RAG:

- ▶ 业务规则**变化快** (规程改版、标准更新);
- ▶ 内容**经常更新** (组内文献库每月新增几十篇);
- ▶ 需要**查用户 / 组织自己的资料** (模型训练时根本没见过)。



第四讲的类比: 开卷答题, 手册
可以随时换新版

需要 $\rightarrow$ **RAG**; 不需要 $\rightarrow$ Prompt 即可

化工判断: “HF 防护要求”在**手册**里 $\rightarrow$ RAG;
“HF 为什么腐蚀玻璃”是**通识** $\rightarrow$ Prompt。

Step 3: 要查数据 / 做动作吗? $\Rightarrow$ Function Call / MCP

- 查数据** 实时位号、库存、订单、物性数据库——模型“猜”不出来, 必须调接口
- 做动作** 生成工单、发送报警、写入记录——这是从“脑力”到“行动力”
- 多个工具** 工具一多 (DCS、LIMS、文献库、计算器 ...), 就用 MCP 统一接口, 便于维护

需要 $\rightarrow$ **Function Call / MCP Tools**; 不需要 $\rightarrow$ 进入下一步

第五讲的判断标准仍然成立:

只要答案会随时间变、且存在某个系统里, 就不要让模型回忆, 让它去查。

Function Calling 文档: api-docs.deepseek.com,
help.aliyun.com/zh/model-studio

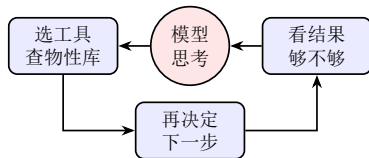
Step 4: 要分步骤执行吗? $\Rightarrow$ Workflow 还是 Agent

Workflow: 步骤由人写死



每步可控、可审计、稳定
开车方案、报警单、检测报告都该这么走

Agent: 步骤由模型自己定



灵活, 但更贵、更慢、更难预测

Workflow 步骤人写死: 每一步可控、可审计、稳定——有固定 SOP 的事都该用它

Agent 步骤模型自己定: 灵活, 但更贵、更慢、更难预测——留给真正开放的问题

一个提醒: 如果你看到模型“推理混乱”, 通常是误用了单轮推理, 而不是模型不聪明。

Step 5: 要统一语气 / 格式吗? ⇒ 微调

这类需求长这样: 客服话术、法律风格、医疗规范、指标说明、协议提示——

化工版: 500 份检测报告的结论段必须一个模板、危险性提示一句都不能漏。

Prompt 能做到 95% 的时候格式对

微调能做到 每一次都对 (Prompt 永远无法 100% 稳定风格)

需要 → 微调; 不需要 → Prompt 就够

但注意顺序: 微调是**最后一步**, 是在 Prompt、RAG、Function Call 都到位、内容已经正确、只剩风格不稳时才做的事——第四部分的案例会演给你看。

Step 6: 要跨任务记忆吗? $\Rightarrow$ Memory / 上下文工程

用户偏好 “我喜欢简短回答” 说过一次, 以后都要记住

历史会话 上周查过三号反应器, 这周不用再报位号

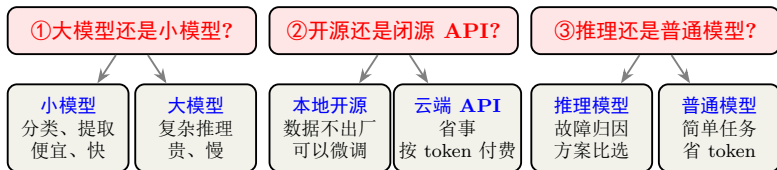
角色设定 值班工程师 vs. 实习生, 回答深度不同

跨轮任务 一篇综述改了 20 轮, 第 21 轮还要知道前面改了什么

需要 $\rightarrow$ **Memory / 向量记忆** (上下文工程); 不需要 $\rightarrow$ 单轮即可

第八讲的结论: “长期记忆” 是一种**工程机制**, 不是模型能力——单靠 Prompt 做不到, RAG 不是为它设计的, 微调更无能为力。

能力选完, 还要选模型: 三个二选一



三个问题分别由任务难度、数据能否出厂、是否多步推导决定
一个系统里可以混用(模型路由, 第二讲)

大 vs. 小

分类、提取、改写用小模型 (便宜、快); 复杂推理、长文综述用大模型。第二讲: 成本 $\approx \text{token} \times \text{激活参数}$

开源 vs. 闭源

数据不能出厂、要本地部署、要微调 → 开源 (Qwen、DeepSeek 开源版); 追求省事 → 云端 API

推理 vs. 非推理

多步推导 (故障归因、方案比选) 用推理模型 (如 DeepSeek-R1); 简单任务用推理模型是浪费 token

判断题 (先举手, 再揭晓)

课堂互动③

1. 组里的文献库每月新增几十篇, 最合适的做法是每月把新文献微调进模型。
2. “把这段英文摘要翻成中文” 这个需求, 用 Agent 做效果会更好。
3. 要让 AI 报出常压塔塔顶的当前温度, Prompt 里写得再详细也没用。
4. 厂里的配方数据不能出厂, 所以只能放弃用大模型。

1. ✗ 知识在变, 走 Step 2 → RAG: 换文档即可, 微调改起来最慢也最贵。
2. ✗ 生成类单步任务, Prompt 就够; 上 Agent 只会更贵、更慢、更不可控。
3. ✓ 实时数据在 DCS 系统里, 走 Step 3 → Function Call, 让模型去查而不是回忆。
4. ✗ 走问题六 → 开源模型本地部署, 数据不出厂照样能用。

七种能力放在一张表里比

能力	改什么	成本	生效	更新	可追溯	讲次
Prompt	输入	几乎为零	立刻	改一句话	弱	3
RAG	输入 + 资料	中	小时级	换文档	✓	4
Function Call	输入 + 系统	中	天级	改接口	✓	5
MCP	工具接入方式	中	天级	加工具	✓	6
Workflow / Agent	执行流程	中 ~ 高	天级	改流程	看设计	7
Memory (上下文工程)	会话状态	中	天级	自动积累	弱	8
微调	参数	高	周级	重新训练	弱	9

看“更新”和“可追溯”两列：需要经常改、需要给出处的，都不该压进参数里。

能用便宜的办法解决，就别上贵的；能追溯的，优先于不能追溯的。

Step 2: 先用 Prompt 跑出 MVP ——这一步不能跳过

你是 XX 客服，请始终保持以下规则：

1. 语气：简洁、礼貌、不啰嗦。
2. 格式：分点回答。
3. 风险类问题必须提示阅读用户协议。
4. 如果需要查询订单，请告诉用户 ``我来为您查询一下''。

几分钟就能跑通原型；你能判断这个任务**适不适合**做 AI，能让业务方**看到效果**，决策效率提高 10 倍。

课堂互动④ · 动手改写 (2 分钟)

把上面这条 Prompt 改成“化工实验室安全助手”的版本：角色、语气、格式各改一条，再加一条化工特有的规则（例如“涉及剧毒品的问题必须提醒查阅 MSDS”）。

写完和同桌交换，找出对方 Prompt 里**做不到**的那一条——它多半就是下一步要加的能力。

Step 3: 业务问答 → 构建 RAG (第四讲)

为什么？因为价格、规则、服务策略**频繁变动**，单靠 Prompt 或一次性微调，答案很快就过时。

实用建议	怎么做
分段并标来源	每段 200 ~ 800 字，记录文档名、版本、更新时间、段落 ID
生成向量入库	每段做 embedding 存进向量库，同时保存原文与元数据
检索策略	向量召回若干段，再按置信度过滤排序 (可结合关键词检索)
拼接与提示	高相关片段按优先级拼进 Prompt，并要求“无相关资料则说明未匹配”
版本与更新	易变内容建立每日 / 每小时增量更新，元数据标注最后更新时间
可追溯	回答里显示来源引用 (至少文档名 + 更新时间)
监控与回收	低置信或被用户打回的回答，收进训练池，供后续微调或规则补丁

化工版：把安全手册、SOP 按“一条规程一段”切，元数据里必须有**版本号**——旧版规程答出来是事故隐患。

Step 4: 加入 Function Call ——让模型能“做事” (第五讲)

```
{
  "name": "queryOrder",
  "description": "根据手机号查询订单状态",
  "parameters": {
    "type": "object",
    "properties": {
      "phone": {"type": "string"}
    },
    "required": ["phone"]
  }
}
```

示例: 查订单状态的接口定义

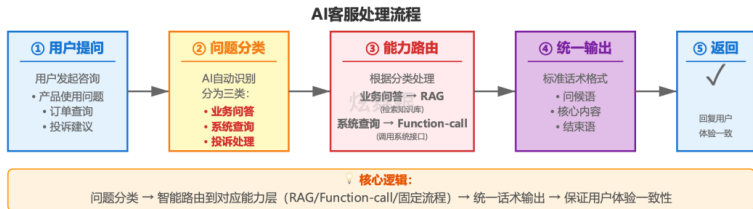
把接口暴露给模型后, 它会自己学会:

- ▶ **何时该调**: “帮我查一下订单发货了吗” → 调 `queryOrder`;
- ▶ **何时不该调**: “你们快递一般多久到” → 不乱查;
- ▶ **怎么说结果**: 接口返回 `shipped` → “您的订单已发货, 正在配送中”。

化工版接口: `query_dcs_tag(tag_id)` 根据位号读实时值;
`query_msds(cas_no)` 按 CAS 号查安全数据表。

Prompt 只能生成内容, RAG 只能补知识, **只有 Function Call 能查实时数据、执行动作** ——从“脑力”到“行动力”。

Step 5: 引入 Workflow ——让任务有逻辑顺序 (第七讲)



用户提问 → 问题分类 → 能力路由 (RAG / Function Call / 固定流程) → 统一输出 → 返回。

Workflow 的本质: 让 AI 不再“自由发挥”, 而是按人写好的业务流程一步步走:

- ▶ 复杂任务拆成有序步骤;
- ▶ 每一步可控、可审计, 不会乱跳;
- ▶ 前后一致, 不忘流程节点。

客服最怕的：跳过验证、误判状态、提前承诺退款。

化工最怕的: 开车方案跳过氮气置换这一步、报警单漏填联锁值。

真正的业务系统都有固定 SOP, SOP 写清楚, Workflow 就定得清楚。

常见 Workflow 工具, 认个脸



LangChain / LlamaIndex 代码级编排框架, 适合有开发能力的团队
Agently 面向 Agent 编排, 支持自然语言定义流程
Dify 可视化拖拽搭 Workflow, 零基础同学期末项目的首选
MCP Workflow 用 MCP 协议统一接工具 (第六讲), 见 github.com/modelcontextprotocol

国内可直接用的低代码平台: 阿里云百炼 的应用编排, 文档见 help.aliyun.com/zh/model-studio。

Step 6 / 7: 风格不稳 → 微调; 记不住用户 → Memory

该微调了的“症状”

- ▶ 语气时而热情、时而冷淡;
- ▶ 同样的问题回答长短不一;
- ▶ 有时漏掉风险提示;
- ▶ 乱改格式或字段, 统一模板无法 100% 落地。

这些不是模型“偶尔不听话”, 而是需要稳定的**认知层行为**——只能靠微调。

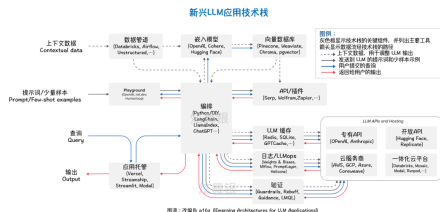
该上 Memory 了的“症状”

- ▶ 用户说过“我喜欢简短回答”, 下一轮就忘;
- ▶ 刚查过订单, 又问一遍手机号;
- ▶ 任务跨多轮 (旅行规划、内容编辑);
- ▶ 长对话要引用前面 20 ~ 30 轮。

记忆是**工程机制**, 不是模型变聪明。

微调的目标不是让模型更聪明, 而是让它**每次输出都一致**: 统一语气、固定格式、固定行为规范、固定拒答规则。

退一步看全局: LLM 应用技术栈



一个真正能落地的 AI 应用, 是 **模型** → **能力层** → **工程层** → **业务层** 的系统工程: 底层是基础模型、Embedding、向量库; 中间是 RAG、Function Call、Workflow、MCP; 再上是编排、日志、验证; 最后才是用户看到的界面。

不是“提示词魔法”能解决所有问题，不是“微调”就能一劳永逸，也不是“开通一个 API”就能上线。

配图: 新兴 LLM 应用技术栈 (改编自 a16z)

课堂互动⑤ · 现场演示: 只用 Prompt 做“安全规程问答”, 看它在哪里露馅

打开 chat.deepseek.com, 用互动④写的 Prompt 问“本实验室氢氟酸操作的防护要求是什么”。全班一起找：哪句是编的？哪句没有出处？缺的正是 RAG 和 Function Call 要补的。

第五部分 · 五个化工需求, 各走一遍决策路

需求	任务类型	关键约束	方案组合	讲次
①文献综述助手	问答 + 生成	文献月月新增, 要引用	Prompt + RAG + Workflow (大模型)	3 4 7
② DCS 异常解释助手	操作 + 推理	秒级实时, 数据不出厂	Function Call + RAG + Workflow (本地开源)	4 5 7
③实验报告写作助手	生成 + 提取	格式固定, 可容错	Prompt + Few-shot + Workflow (小模型)	3 7
④安全规程问答	问答	零编造, 版本要对	RAG + Prompt 拒答规则 (+ 微调, 可选)	3 4 9
⑤配方筛选建议	推理 + 操作	要调物性库和计算工具	MCP Tools + Agent + 推理模型	5 6 7

五个需求, 五种组合——没有一个是“一种能力包打天下”, 也没有一个从微调开始。

接下来逐个看它们是怎么“走”出来的, 你可以直接拿去当期末项目的选题。

需求①文献综述助手 / 需求③实验报告写作助手

①文献综述助手

“近三年渣油加氢催化剂失活的结论有哪些?”

- ▶ Step 1: 问答 + 生成 → 主线 RAG
- ▶ Step 2: 文献月月新增 → **RAG**, 切块带 DOI
- ▶ Step 3: 不查系统 → 跳过
- ▶ Step 4: 检索 → 分组归纳 → 成文, **Workflow** 三步
- ▶ Step 5 / 6: 不需要
- ▶ 模型: 长文归纳用**大模型**, 云端 API 即可 (公开文献)

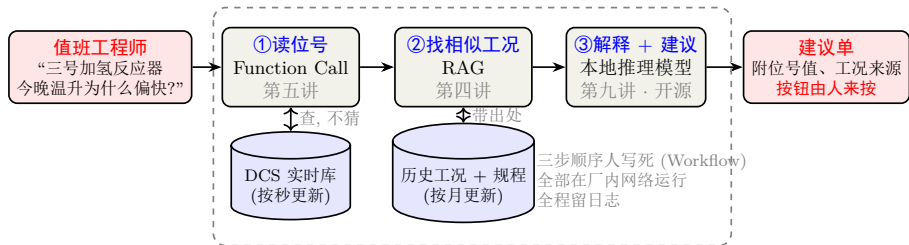
③实验报告写作助手

“把这次精馏实验的原始数据和记录, 写成规范报告。”

- ▶ Step 1: 生成 + 提取 → 主线 Prompt
- ▶ Step 2: 报告格式几年不变 → 不用 RAG
- ▶ Step 3: 数据学生自己贴 → 不用接口
- ▶ Step 4: 提数据 → 算回流比 → 套模板, **Workflow**
- ▶ Step 5: 格式靠 **Few-shot** 给两份范例就够
- ▶ 模型: **小模型**, 省钱; 计算部分交给代码而不是模型

两个需求都是“轻方案”: 一下午能跑通, 成本几十元。
核对仍是你的责任: 综述里每条引用都要点开看一眼。

需求② DCS 异常解释助手: 六步走完, 方案长这样



为什么不微调

温度按秒变、工况按月变,微调进去的知识
改不动 (Step 2)

为什么 Workflow 不用 Agent

“先查数、再找相似工况、再解释”是固定套路, 写死更稳 (Step 4)

为什么本地开源

DCS 数据不出厂 (问题六); 推理部分可用 DeepSeek-R1 开源本地跑

输出只到“建议”为止: 降进料量的按钮, 永远由值班工程师来按。

需求④安全规程问答 / 需求⑤配方筛选建议

④安全规程问答

“使用氢氟酸的防护要求是什么？”

- ▶ Step 1: 问答类 → 主线 **RAG**
- ▶ Step 2: 规程会改版 → RAG, 元数据带**版本号**
- ▶ Step 3 / 4 / 6: 不需要
- ▶ Step 5: 拒答规则必须稳定——先靠 Prompt “无出处不答”；上线后若仍偶尔编造, 再考虑**微调**
- ▶ 准确率: 零编造, 每条回答附规程条款号

⑤配方筛选建议

“从 30 种候选溶剂里, 挑出适合萃取该组分的前三名。”

- ▶ Step 1: 推理 + 操作 → 主线 **Agent**
- ▶ Step 3: 要查物性库、调相平衡计算 → **MCP Tools** 统一接入
- ▶ Step 4: 筛选路径不固定 (先按沸点还是先按毒性?) → Agent 自己决定
- ▶ 模型: 多步推导用**推理模型**; 每一步的计算交给工具, 不让模型心算
- ▶ 结果是“建议”, 最终配方要实验验证

④是最典型的“要出处”场景, ⑤是全程唯一真正需要 Agent 的场景——
决定用不用 Agent 的, 是“路径是否固定”, 不是“问题够不够高级”。

小组：抽一个化工需求，现场出方案，交叉互评

课堂互动⑥

每组抽一张需求卡 (备选：催化剂制备 SOP 助手 / 装置巡检报告生成 / 物性数据查询助手 / 工艺参数优化建议 / 新员工安全培训问答 / 专利检索助手)。

6 分钟：填六问 → 走六步 → 写出“能力组合 + 模型选型 + 分三阶段怎么建”。
然后与相邻组交换方案，用下面的评分表打分，各推荐 1 组上台 2 分钟。

评分项	看什么	分
需求拆得开	六问是否都有答案，任务类型是否拆成子任务	3
能力选得对	每个能力都能说出“哪一步、为什么需要”	3
没有过度设计	有没有多余的微调 / Agent；是否从 Prompt MVP 起步	2
安全与核对	数据出不出厂、谁来按按钮、出处能否核对	2

够用就好：负责任地选择和使用 AI

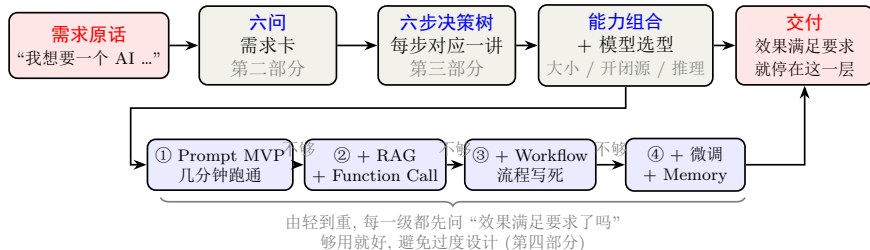
前面九讲学到的能力越多，越容易“什么都想用上”。今天的方法论其实在教一种克制：

- ▶ **由轻到重**：先用 Prompt 跑通，效果不够再加一层，不为炫技上 Agent 和微调；
- ▶ **算清成本**：每加一层都问一句“多花的钱和时间，换来了什么”；
- ▶ **核对与追溯**：优先选能给出处的方案；AI 的结论进入报告前，自己点开来源看一眼；
- ▶ **守住边界**：厂内数据不随手上传公有云；涉及安全的动作，AI 只建议，人来执行；
- ▶ **诚实署名**：期末项目里哪些是 AI 生成、哪些是自己验证过的，写清楚。

Q:讨论 (2 分钟)：你在互动⑥的方案里，有没有一层是“可以不加”的？去掉它，项目会损失什么？

Q:回到互动①：现在再投一次票，三道题的答案和开场时一样吗？

本讲总结：从需求到方案的一条路



先填六问，再走六步，得到能力组合和模型选型；然后按由轻到重的顺序分阶段建，

每一级都先问“效果满足要求了吗”——满足就交付，不满足再加一层。

随堂快测

课堂互动 · 3 道题

- Q1 (单选) 需求: “让 AI 报出常减压装置常压塔塔顶的当前温度并判断是否超限。” 最不可缺少的能力是?
A. 微调 B. Function Call C. Agent D. Few-shot
- Q2 (判断) 只要问题足够复杂, 就应该优先用 Agent 而不是 Workflow。
- Q3 (简答) 一位同学想把组里每月更新的文献库微调进模型, 做一个“文献问答助手”。用六步决策树指出问题, 并给出你的方案组合。

答案: Q1 选 **B** (实时数据在 DCS 系统里, 必须现查);

Q2 **✗** 错误, 决定因素是**路径是否固定**: 路径固定用 Workflow, 开放问题才用 Agent;

Q3 Step 2 “知识经常更新” → 应选 RAG 而非微调; 方案: Prompt + RAG (切块带来源) + 大模型 API。

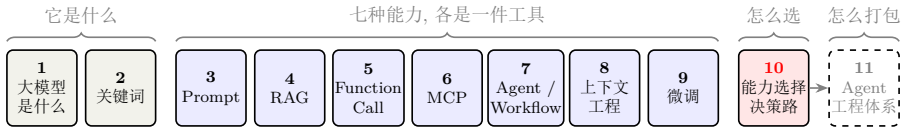
线上提交, 当堂出分。

课后任务与拓展

- 必做① **需求卡 + 决策路**: 从互动⑥的六个化工需求里另选一个 (不能选本组抽到的), 填六问、走六步, 写出能力组合、模型选型和三阶段建设计划 (一页 A4)。
- 必做② **反例分析**: 找一个你见过或用过的 AI 应用, 用本讲的表判断它是否“过度设计”或“能力缺失”, 说明依据 (300 字)。
- 选做③ **动手搭**: 在 [阿里云百炼](#) 用应用编排搭出需求③“实验报告写作助手”的 Prompt + Workflow 版本, 截图并附一次真实运行结果。

拓展阅读: [百炼文档](#) 的 RAG 知识库、Function Call、Agent、微调章节 (对照本讲的六步各看一节);
[MCP 协议仓库](#) (选读)。

回望前十讲：工具箱已经齐了，还差最后一步



第一、二讲说清“它是什么”；第三到九讲逐个交给你七种能力；第十讲把它们串成一条决策路——从今天起，你拿到任何一个 AI 需求，都知道从哪里开始问。

但还有一个问题没回答：选好的 Prompt、工具和流程，怎么让下一个项目、下一个人不用从头再来？这就是第十一讲要讲的工程体系。

配图：前九讲的知识全景

下讲预告：围绕 Prompt 形成的 Agent 工程体系

- 今天选好了能力组合, 然后呢 一条好 Prompt、一套工具、一段流程, 怎么让下个项目、下个人不用从头再写
- Skill 是什么 把成熟的 Prompt + 步骤 + 示例装进一个可复用的“技能包”, 需要时才加载
- Harness 是什么 围着模型搭的“驾驭架”: 工具、权限、循环、检查、日志, 让 Agent 稳定地跑
- Agent Plugins 是什么 把 Skill、工具、配置打成插件, 装到别人的 Agent 里就能用, 经验得以沉淀和分发
- 和前十讲的关系 第三讲的 Prompt、第五六讲的工具、第七讲的 Agent, 在第十一讲被“打包”成工程体系
- Q: 课前思考: 你在互动⑥里写的方案中, 哪些部分换一个化工需求还能原样复用? 把它们列成一张清单带来。
第十一讲为本课程最后一讲。

Thank
You!

 Your opinion
Matters

第十讲 · AI 能力选择指南——从需求到方案的完整决策路
周天航 · 中国石油大学 (北京)

下课。请完成课后任务,下次课把你的需求卡和决策路带来,我们看看哪些部分能“打包”复用。